# Combining Mamba Mixture-of-Experts (MoE) and Chain-of-Thought (CoT) Distillation


**TEAM MEMBERS**

Matthew Reynolds (mcr2235)
Jacob Boyar (jmb2536)
Eric Yi (hy2646)
Zara Iqbal (zi2141)


**CODE AND PROJECT LINK**

https://colab.research.google.com/drive/1a6557oXASDlFRWZn8O6MEGbBto60jKf7?usp=sharing

**VIDEO LINK**

https://youtu.be/NruJO_AlAe4


**ABSTRACT**

Though transformers are the traditional architecture used in LLMs, other architectures have been introduced to provide alternative benefits. Among them, Mamba was created to improve upon the transformer's context based reasoning over long sequences. It is a model combining structured state space sequence models and selective state spaces, and it was found to outperform Transformers of the same size and match transformers twice its size. Further improvements to Mamba have included implementation of Chain-of-Thought distillation - teaching active recall and reasoning over long contexts - and Mixture of Experts - which has already been shown to significantly improve Transformer-based LLMs. However, no study has yet attempted to combine the two methods. This paper looks to rectify that, creating one unified Mamba with both Chain-of-Thought and Mixture of Experts implementation.


**INTRODUCTION & MOTIVATION**

Our initial idea for this project came from the idea of pushing the boundaries of what LLMs can do. However, much of modern research (and the majority of our class) has focused on Transformers as the classical LLM architecture. Rather than attempt to improve upon a highly popular LLM architecture, we thought it would be more interesting to take the path less traveled and focus on Mamba. First created by Albert Gu at Carnegie Mellon University, it has consistently been shown to be a powerful alternative to traditional Transformer architecture. Multiple research papers have already used it as a base to implement further improvements, with both Chain-of-Thought and Mixture of Experts. Given that both of these methods were listed under the 'Research-focused' section of the final assignment, we thought that it would be appropriate to combine the two for our final project. Finally, since two separate research groups have already shown that the model is capable of supporting both of these methods we figured that this assignment would be doable within our allowed timeframe.

## RELATED WORK

Mamba, first detailed in a paper by Albert Gu and Tri Dao, is a homogenous architecture design that combines both structured state space sequence models (SSMs) and selective state spaces. Both of these methods are alternatives to the traditional Transformer architecture, developed to combat its computational inadequacy on long sequences. Individually, however, these models perform poorly in regards to attention, language, and their ability to make content-based reasoning. Mamba was created by integrating selective SSMs into an end-to-end neural network architecture, without attention or MLP blocks. It features fast inference, linear scaling in sequence length, and consistent performance improvements with increasing size in data. Mamba's core strength is its ability to efficiently filter out irrelevant information while remembering important information indefinitely. It has even been shown to outperform transformers on language modeling tasks.

Further research has been done to improve Mamba's capabilities. One study, by Jun-Yu Ma and Tianqing Fang, found that Mamba's theoretically infinite context potential was limited when sequences exceeded training lengths. To combat this, they implemented Chain-of-Thought (CoT) distillation. This teaches Mamba to actively recall and reason over long contexts. They found that, with CoT implementation, Mamba had boosted long-context performance compared to other transformer/hybrid baselines, preserved short-context capabilities, all without any change to the underlying structure of the model.

Additionally, another group of researchers - among them, Maciej Pioro and Kamil Ciebiera - thought to combine the Mamba state space model with Mixture of Experts (MoE). MoE has been shown to significantly improve transformer-based Large Language Models (LLMs). It is a combination of techniques that enables an increase in model parameters with only minimal impact on computation. As such, it can be scaled up to trillions of parameters, and is now routinely used in LLMs. This paper found that, by combining Mamba with MoE, it was able to outperform both Mamba and baseline Transformer-MoE. It reached the same performance as Mamba in 2.35x fewer training steps, all while preserving the inference performance gains of Mamba against Transformer.

## METHODOLOGY / SYSTEM DESIGN

Our MambaMoEClassifier integrates Mamba with MoE and CoT learning into a unified architecture. The model processes input through an embedding layer, followed by two Mamba blocks, a single MoE layer, two more Mamba blocks, and finally two classification heads - one for the main task and one for CoT learning.

For the Mamba blocks, we used the official mamba-ssm library to ensure correct implementation of the selective scan algorithm. Each block implements input-dependent

parameters through linear projections of the input, enabling content-aware state updates. We configured the blocks with a state dimension of 32 and a local convolution window of 4, following standard practice from the original Mamba paper.

Our MoE layer uses a gating network that routes each token to its top-k=2 experts from a pool of 8. Each expert is a two-layer MLP with an expansion factor of 4. To encourage balanced expert utilization, we included an auxiliary load-balancing loss that penalizes uneven routing distributions. Unlike the MoE-Mamba paper, which interleaved MoE throughout the model, we placed a single MoE layer after the second Mamba block. This was done to balance expressiveness with computational cost, given our resource constraints.

For CoT, we took a simplified approach compared to the original paper by Ma and Fang. Rather than distilling from a teacher model (which would require significant computation), we created pseudo-labels based on sequence complexity. Sequences above median length were labeled as "requiring extended reasoning," while shorter sequences were labeled as "straightforward." A separate thought head predicts this auxiliary target, encouraging the model to develop internal representations that distinguish between simple and complex inputs.

Our total loss combines three components: the main cross-entropy loss for sentiment classification, the CoT auxiliary loss (weighted at 0.2), and the MoE load-balancing loss (weighted at 0.05). This formulation allows the primary task to dominate while still providing a learning signal for expert utilization and reasoning behavior.

**EVALUATION / RESULTS**

We evaluated our model on the Stanford Sentiment Treebank binary classification task (SST-2), which contains roughly 67,000 training samples and 872 validation samples. We used BERT's tokenizer with a maximum sequence length of 128 tokens. Training was initially set for 3 epochs but was extended to 5 epochs to allow more time for convergence. We used the AdamW optimizer with a learning rate of 5e-5 and Automatic Mixed Precision (AMP) for efficiency on a T4 GPU environment.

During experimentation, we also adjusted the MoE load-balancing loss weight (beta) from 0.01 to 0.05 to encourage more even expert utilization. Based on the training run over 5 epochs, the following trends were observed:

| Epoch | Train Loss | Val Loss | Accuracy | F1 Score | MoE Util (%) |
|---|---|---|---|---|---|
| 1 | 0.6804 | 0.6524 | 0.6193 | 0.6191 | 0 |
| 2 | 0.6066 | 0.6259 | 0.6502 | 0.6262 | 28.29 |
| 3 | 0.5336 | 0.5445 | 0.7328 | 0.7325 | 31.47 |
| 4 | 0.4718 | 0.5388 | 0.7374 | 0.7366 | 4.43 |
| 5 | 0.4163 | 0.5404 | 0.7248 | 0.7246 | 29.59 |

*Table 1: Training results*

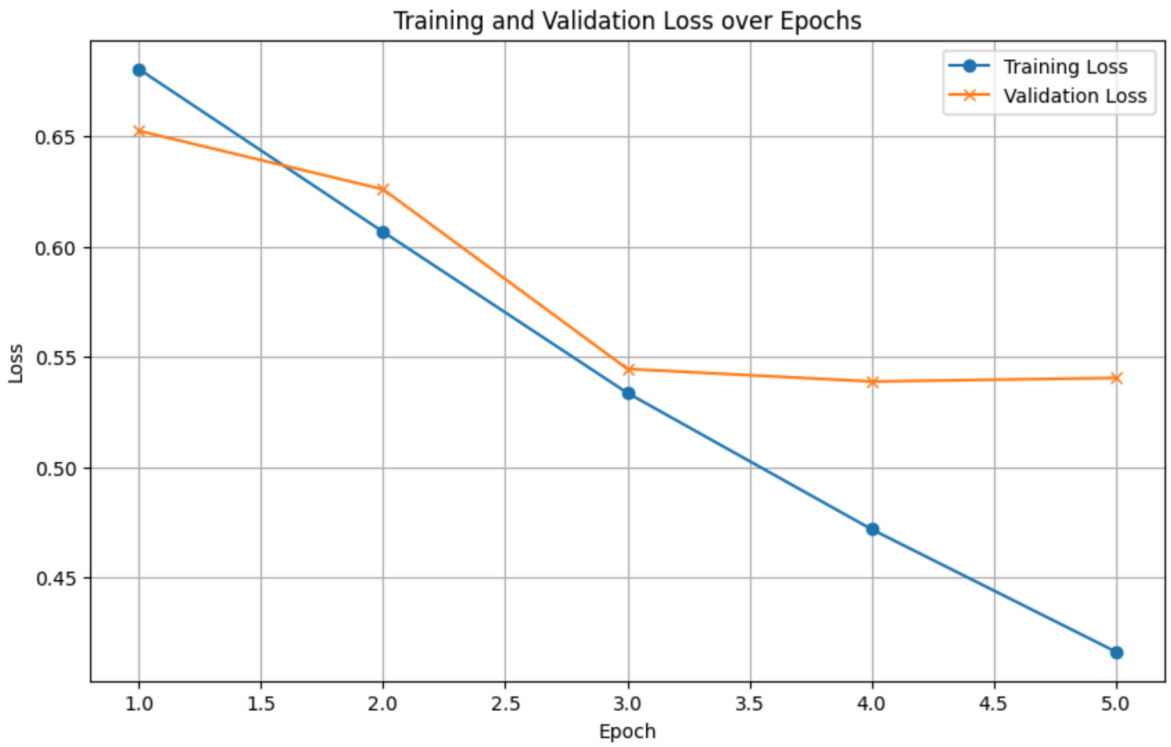


*Figure 1: Training and Validation Loss*

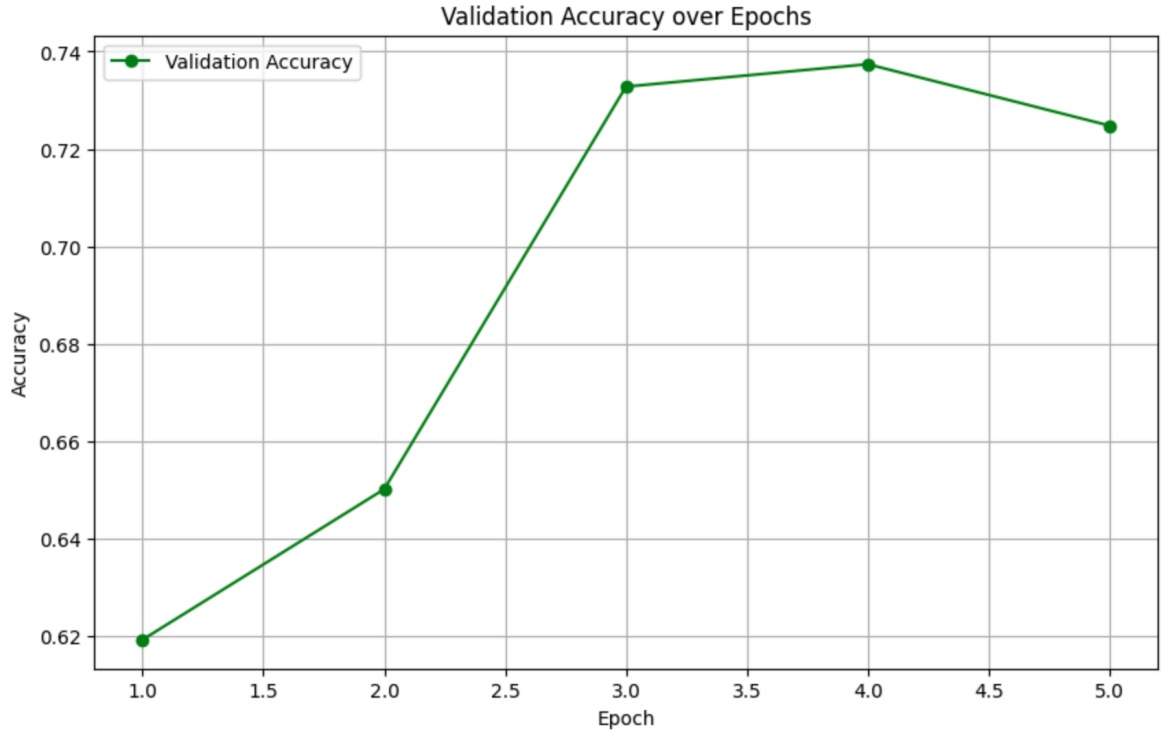


*Figure 2: Validation Accuracy*

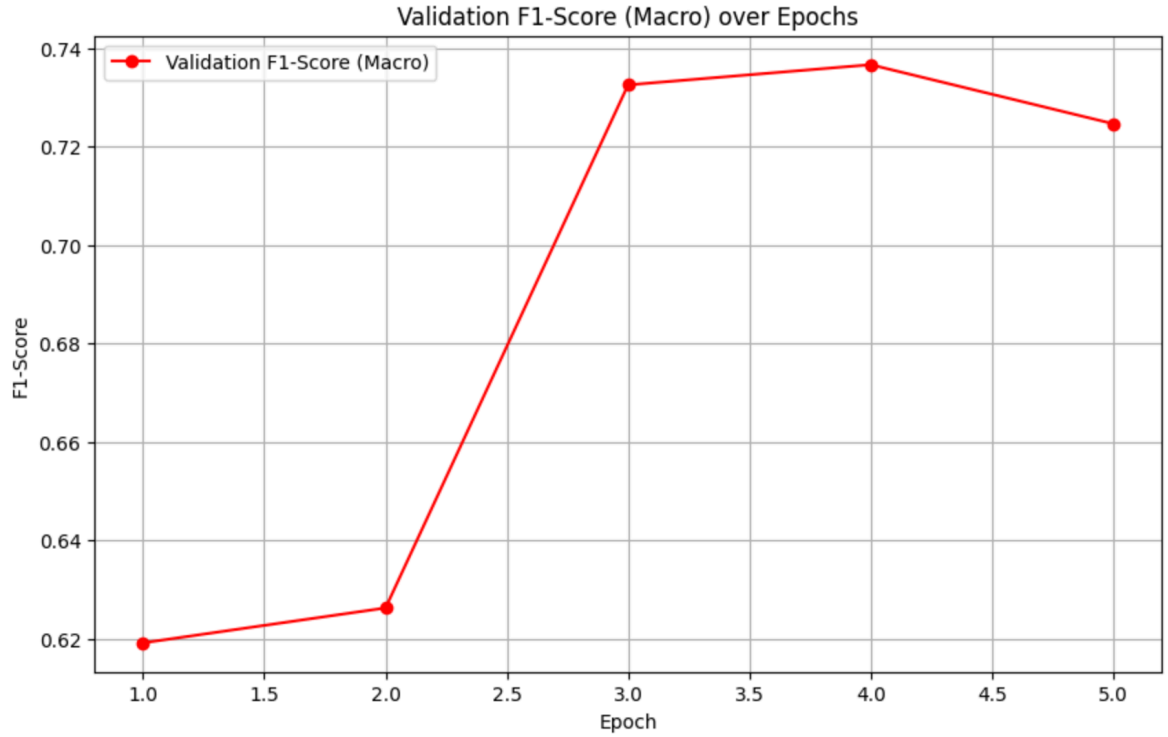


*Figure 3: Validation F-1 Score*

Training loss decreased consistently from 0.68 to 0.42 across all epochs, indicating successful optimization. Validation loss initially decreased but plateaued after epoch 3, with a slight increase in the final epoch, suggesting the onset of overfitting. Peak performance was achieved at epoch 4 with 73.74% accuracy and 0.7366 F1 Score.

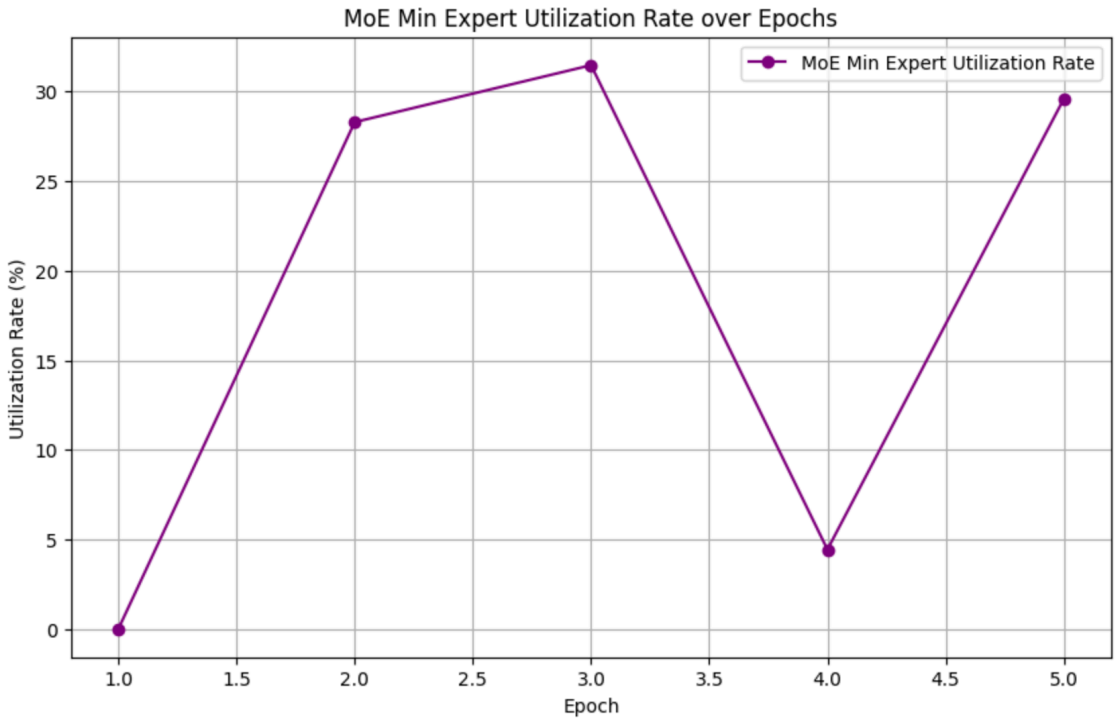


*Figure 4: Minimum Expert Utilization Rate in MoE*

A striking finding is the volatility of MoE expert utilization across epochs. The minimum expert utilization fluctuated dramatically throughout training: 0% in epoch 1, 28.3% in epoch 2, 31.5% in epoch 3, then dropping to just 4.4% in epoch 4, and back up to 29.6% in epoch 5. Interestingly, our best validation performance coincided with the lowest expert utilization, suggesting a potential inverse relationship between uniform expert usage and task performance. This may indicate that for binary sentiment classification, the model benefits from specializing a small subset of experts rather than distributing computation uniformly.

It is worth noting that our implementation operates at a fundamentally different scale than the reference papers. MoE-Mamba was evaluated at 25M-790M parameters on billions of tokens, while we used approximately 500K parameters on 67K samples. Similarly, the CoT paper by Ma and Fang used a Llama-3.1-8B-Instruct teacher model and evaluated on contexts up to 100K tokens. These differences limit direct comparison of techniques, but our results still demonstrate that the core concepts can be combined successfully.

## DISCUSSION & LIMITATIONS

Several components of our implementation worked well. The environment setup - including installation of transformers, datasets, and mamba-ssm was successful. The data pipeline, including loading and preprocessing of SST-2, tokenization, and creation of PyTorch DataLoaders with pseudo-labeling for CoT, worked as intended. The custom MambaMoEClassifier model was successfully defined and instantiated, integrating Mamba blocks, the MoE layer, and dual classification heads. The implementation of AMP with torch.autocast and GradScaler was seamless, allowing for efficient training on GPU. Overall, the training and evaluation loops functioned correctly, and the model achieved reasonable performance on sentiment classification, demonstrating the potential of the MambaMoE architecture.

However, the most significant challenge was the erratic behavior of the MoE Min Expert Utilization Rate. The sharp fluctuations, particularly the dip to 4.43% in Epoch 4, indicate that the current gating mechanism and load-balancing loss might not be effectively ensuring uniform expert usage. Even after adjusting beta from 0.01 to 0.05, this volatility persisted. This could lead to some experts being underutilized, limiting the benefits of the MoE architecture.

Additionally, our implementation operates at a fundamentally different scale than the reference papers. MoE-Mamba was evaluated at 25M-790M parameters on billions of tokens, while we used approximately 500K parameters on 67K samples. Our pseudo-label approach to Chain-of-Thought is also a significant simplification compared to true reasoning distillation with a teacher model. Finally, the slight increase in validation loss and decrease in accuracy in the final epoch suggest the onset of overfitting, which could be addressed with regularization techniques or early stopping in future work.

## CONCLUSIONS & FUTURE WORK

We presented MambaMoECoT, a proof-of-concept architecture demonstrating that selective state spaces, sparse expert routing, and chain-of-thought inspired multi-task learning can be combined for text classification. Our experiments on SST-2 achieved 73.74% accuracy while revealing both the promise and challenges of this hybrid approach.

The primary contribution of this work is establishing that these three techniques - each addressing different aspects of model efficiency and capability - can coexist in a unified architecture. Our environment setup, data pipeline, model instantiation, and AMP implementation all functioned as intended, demonstrating the overall viability of the MambaMoE architecture. While our implementation makes significant simplifications compared to the original papers, it provides a foundation for more sophisticated combinations.

Several promising directions emerge from this work. The most pressing issue is the MoE expert utilization volatility. Even after adjusting beta from 0.01 to 0.05, the Min Expert Utilization Rate fluctuated dramatically across epochs, dropping as low as 4.43% in Epoch 4.

Future work should focus on refining the gating mechanism and exploring alternative load-balancing strategies such as router z-loss or expert capacity constraints to ensure more consistent expert engagement.

Additionally, the slight increase in validation loss in the final epoch suggests the onset of overfitting. Implementing regularization techniques (such as dropout or increased weight decay), early stopping, or more sophisticated learning rate schedules could help improve generalization.

Beyond addressing these immediate challenges, we see several longer-term directions. First, implementing true CoT distillation with a teacher model generating reasoning traces could unlock the full potential of the auxiliary task. Second, exploring different MoE placement strategies (interleaved vs. concentrated) may improve expert utilization. Third, evaluating on longer-context tasks would better leverage Mamba's linear complexity advantage. Finally, scaling experiments to larger models and datasets would enable meaningful comparison with the reference papers. Ablation studies isolating the contribution of each component (Mamba-only, Mamba+MoE, Mamba+CoT) would also clarify their individual and combined effects.